

Numerical resolution of Steady State Stream Function-Vorticity Formulation-Navier Stokes Equation

Tags: #CFD #Navier-Stokes-Equation

Introduction

In this note, we are going to discuss how to resolve a problem of steady state stream function-vorticity formulation with pseudo-time marching method.

Pseudo-time marching method for naive correction

Idea

Let us recall the stream function-vorticity formulation of steady state Navier-Stokes equation:

$$\begin{cases} \nu \nabla^2 \omega = \mathbf{u} \cdot \nabla \omega \\ \nabla^2 \psi = \omega \\ \mathbf{u} = -\nabla \times \psi \mathbf{e}_z \end{cases}$$

where ψ is the stream function and $\mathbf{e}_z$ is the z -axis of the coordinate system, and $\mathbf{u}$ is the velocity.

To solve this system with the prescribed boundary conditions of ψ , we will iterate the boundary value of ω till the Neumann condition $\frac{\partial \psi}{\partial n}$ is satisfied. For the first iteration, we consider the problem as a Stokes problem. After solving that, we use the form of non-stationary formulation of the Navier-Stokes equation such that

$$\frac{\partial \omega}{\partial t} + \mathbf{u}_f \cdot \nabla \omega = \nu \nabla^2 \omega$$

where $\mathbf{u}_f$ is the frozen velocity. The discrete form is given by the following expression

$$\frac{\omega^{t+\Delta t} - \omega^t}{\Delta t} + ((\mathbf{u}_f \cdot \nabla) \omega)^t = \nu (\nabla^2 \omega)^{t+\Delta t}$$

where t is the artificial time.

We introduce an intermediate value ω^* such that

$$\begin{aligned}\frac{\omega^* - \omega^t}{\Delta t} + ((\mathbf{u}_f \cdot \nabla)\omega)^t &= 0 \\ \frac{\omega^{t+\Delta t} - \omega^*}{\Delta t} &= \nu(\nabla^2 \omega)^{t+\Delta t}\end{aligned}$$

these equations can be reformulated as

$$\begin{aligned}\omega^* &= \omega^t - \Delta t((\mathbf{u}_f \cdot \nabla)\omega)^t \\ -\omega^{t+\Delta t} + \nu \Delta t \nabla^2 \omega^{t+\Delta t} &= -\omega^*\end{aligned}$$

and we want that

$$\lim_{t \rightarrow \infty} \frac{\|\omega^{t+\Delta t} - \omega^t\|_{L^2}}{\Delta t} \rightarrow 0$$

By setting

$$t = t_0 + m\Delta t \implies t + \Delta t = t_0 + (m+1)\Delta t, \quad m \in \mathbb{N}$$

we can use the following notation

$$(\omega^n)^t = \omega^{(n,m)}, \quad (\omega^n)^{t+\Delta t} = \omega^{(n,m+1)}$$

Thus the numerical scheme can be summarized as the following steps:

1. Solve Stokes problem in the first iteration.
2. Using the solution of the Stokes problem, set the initial velocity and the initial vorticity

$$\begin{aligned}\mathbf{u}_f^{(0)} &= \mathbf{u}_0 \\ \omega^{(0,0)} &= \omega_{Stokes} \\ \omega_{BC}^{(0)} &= \omega_{BC_{Stokes}}\end{aligned}$$

3. Apply this velocity to determine the intermediate value ω^* and the interior vorticity update

$$\begin{aligned}(\omega^*)^{(n,m)} &= \omega^{(n,m)} - \Delta t((\mathbf{u}_f^{(n)} \cdot \nabla)\omega^{(n,m)}) \\ \omega^{(n,m+1)} - \Delta t(\nu \nabla^2 \omega^{(n,m+1)}) &= -(\omega^*)^{(n,m)}\end{aligned}$$

which needs to satisfy the condition

$$\lim_{m \rightarrow \infty} \frac{\|\omega^{(n,m+1)} - \omega^{(n,m)}\|_{L^2}}{\Delta t} \rightarrow 0$$

while using the boundary vorticity in the loop with respect to time, i.e

$$\omega^{(n,m+1)}|_{\partial\Omega} = \omega_{BC}^{(n)}$$

for all pseudo-time steps.

4. Solve the Poisson equation

$$\nabla^2 \psi^{(n)} = \omega^{(n,*)}$$

and compute the difference (error) of the Neumann condition

$$\varepsilon_i^{(n)} = \left(\frac{\partial \psi^{(n)}}{\partial n} \right)_i - (\mathbf{u}_\Gamma \cdot \mathbf{t})_i$$

where $i = 1, 2, \dots, N_{BC}$ is the index of boundary cells.

5. The velocity update is given by

$$\mathbf{u}_f^{(n+1)} = -\nabla \times (\psi^{(n)}) \mathbf{e}_z$$

During the pseudo-time marching at the n -th outer iteration, $u^{(n)}$ is frozen. it is updated only after the Poisson problem for $\psi^{(n)}$ is solved. Let the boundary vorticity be updated with the scheme of naive iterative correction

$$\omega_{BC_i}^{(n+1)} = \omega_{BC_i}^{(n)} + \lambda_h 4\varepsilon_i^{(n)}$$

where λ_h is a relaxation number with respect to the mesh size. And we also have

$$\begin{aligned} \omega^{(n+1,0)} &= \omega^{(n,m)} \\ \omega^{(n+1,*)}|_{\partial\Omega} &= \omega_{BC}^{(n+1)} \end{aligned}$$

About the nonlinear term

- We can use the first-order upwind scheme to discretize the advection term. Upwind schemes use information from the direction where the flow comes from, for example

$$\frac{\partial q}{\partial t} + a \frac{\partial q}{\partial x} = 0$$

Then

$$\begin{aligned} a > 0 : \quad \frac{\partial q}{\partial x} &\approx \frac{q_i - q_{i-1}}{\Delta x} \\ a < 0 : \quad \frac{\partial q}{\partial x} &\approx \frac{q_{i+1} - q_i}{\Delta x} \end{aligned}$$

- We can also use *bcg.h* to compute the advection term in Basilisk.

Thought

- The naive boundary vorticity correction is easy to implement because the update scheme is derived from the Taylor series of the stream function near the boundary, and it is independent of the equation of the vorticity; for a given boundary vorticity and a frozen velocity field, the pseudo-time marching procedure can be considered as an initial value problem.
- We can also solve the advection-diffusion equation instead of using the time marching

while preserving the boundary vorticity update scheme.

- We use this time marching scheme because the advection-diffusion equation solver of the direct resolution uses the first order upwind scheme, and the numerical value diverges with low Reynolds numbers.

Here is the solver of Basilisk :

[[Steady State Solver of Navier-Stokes equation-Naive Correction Time Marching]]

Adjoint State Method

1. Adjoint state method with frozen velocity and pseudo-time marching

Idea

The adjoint equations for steady state Stokes flow are given by

$$\begin{aligned}\frac{\partial \tilde{\psi}}{\partial n} - \mathbf{u}_\Gamma \cdot \mathbf{t} + \beta &= 0 \quad \text{on } \partial\Omega \\ \gamma &= 0 \quad \text{on } \partial\Omega \\ \nabla^2 \beta &= 0 \quad \text{in } \Omega \\ \nabla^2 \gamma &= \beta \quad \text{in } \Omega\end{aligned}$$

and the gradient of the objective functional can be written as

$$\nabla_\eta J = -\frac{\partial \gamma}{\partial n}$$

when we do not apply the regularization of control variable η in the objective functional. Similarly, the adjoint equations for the Navier-Stokes equation with frozen velocity $\mathbf{u}_f$ are given by

$$\begin{aligned}\frac{\partial \tilde{\psi}}{\partial n} - \mathbf{u}_\Gamma \cdot \mathbf{t} + \beta &= 0 \quad \text{on } \partial\Omega \\ \nabla^2 \beta &= 0 \quad \text{in } \Omega \\ \nabla^2 \gamma + Re(\mathbf{u}_f \cdot \nabla) \gamma - \beta &= 0 \quad \text{in } \Omega \\ \gamma &= 0 \quad \text{on } \partial\Omega\end{aligned}$$

and the gradient of the objective functional is

$$\nabla_\eta J = -\frac{\partial \gamma}{\partial n}$$

The numerical scheme can be summarized as the following steps:

1. Solve Stokes problem in the first iteration.

- Using the solution of the Stokes problem, set the initial velocity and the initial vorticity

$$\begin{aligned}\mathbf{u}^{(0,0)} &= \mathbf{u}_0 \\ \omega^{(0,0)} &= \omega_{Stokes} \\ \omega_{BC}^{(0)} &= \omega_{BC_{Stokes}}\end{aligned}$$

- Apply this velocity to determine the intermediate value ω^* and the interior vorticity update

$$\begin{aligned}(\omega^*)^{(n,m)} &= \omega^{(n,m)} - \Delta t((\mathbf{u}_f^{(n)} \cdot \nabla)\omega^{(n,m)}) \\ \omega^{(n,m+1)} - \Delta t(\nu \nabla^2 \omega^{(n,m+1)}) &= -(\omega^*)^{(n,m)}\end{aligned}$$

which needs to satisfy the condition

$$\lim_{m \rightarrow \infty} \frac{\|\omega^{(n,m+1)} - \omega^{(n,m)}\|_{L^2}}{\Delta t} \rightarrow 0$$

while using the boundary vorticity in the loop with respect to time, i.e

$$\omega^{(n,m+1)}|_{\partial\Omega} = \omega_{BC}^{(n)}$$

for all pseudo-time steps.

- Solve the Poisson equation

$$\nabla^2 \psi^{(n)} = \omega^{(n,*)}$$

and use this $\psi^{(n)}$ to update the frozen velocity $\mathbf{u}_f^{(n+1)}$. Then use this frozen velocity to resolve the adjoint equations. However, since we need to solve the advection-diffusion equation of γ , we can use the time marching strategy once again: we assume that

$$\frac{\partial \gamma}{\partial t} - (\mathbf{u}_f \cdot \nabla)\gamma = \nu \nabla^2 \gamma - \nu \beta$$

Thus

$$\begin{aligned}\gamma^* &= \gamma^{(n,m)} + \Delta t(\mathbf{u}_f^{(n)} \cdot \nabla)\gamma^{(n,m)} - \nu \Delta t \beta^{(n)} \\ \Delta t \nabla^2 \gamma^{(n,m+1)} - \gamma^{(n,m+1)} &= -\gamma^*\end{aligned}$$

We can solve this system successively without directly solving the convection-diffusion equation.

- Update the boundary vorticity such that

$$\omega_{BC}^{(n+1)} = \omega_{BC}^{(n)} + \lambda \nabla_\eta J^{(n)}$$

The update of the interior vorticity field is given by

$$\begin{aligned}\omega^{(n+1,0)} &= \omega^{(n,m)} \\ \omega^{(n+1,*)}|_{\partial\Omega} &= \omega_{BC}^{(n+1)}\end{aligned}$$

The update of fixed velocity is given by

$$\mathbf{u}_f^{(n+1)} = -\nabla \times (\psi^{(n)}) \mathbf{e}_z$$

6. Iterates the previous steps till the boundary error is small enough.

Remark

We have two options to update frozen velocity

1. Two levels iteration scheme: In each iteration, we update $\mathbf{u}_f$ after updating ψ . *However, the numerical experiments show that this scheme is sensitive when the mesh resolution increases.* (LDC_adjoint_NS.pdf)
2. Three levels iteration scheme: In each iteration, for a frozen velocity, we iterates the boundary vorticity till the boundary error is small enough, then we update the frozen velocity. (LDC_adjoint_NS2.pdf)

2. Adjoint state method with frozen velocity without time marching

Idea

The adjoint equations of the stream function- vorticity formulation are given by

$$\begin{aligned} \frac{\partial \tilde{\psi}}{\partial n} - \mathbf{u}_\Gamma \cdot \mathbf{t} + \beta &= 0 \quad \text{on } \partial\Omega \\ \nabla^2 \beta &= 0 \quad \text{in } \Omega \\ \nabla^2 \gamma + Re(u_0 \cdot \nabla \gamma) - \beta &= 0 \quad \text{in } \Omega \\ \gamma &= 0 \quad \text{on } \partial\Omega \end{aligned}$$

and the gradient of the objective functional is

$$\nabla_\eta J = \alpha \eta - \frac{\partial \gamma}{\partial n}$$

Assuming that the variables of n -th iteration are known , and the solution is obtained at $n+1$ -th iteration, thus we will have

$$\alpha \eta^{(n+1)} - \frac{\partial \gamma^{(n+1)}}{\partial n} = 0$$

and

$$\begin{aligned} \frac{\partial \tilde{\psi}^{(i)}}{\partial n} - \mathbf{u}_\Gamma \cdot \mathbf{t} + \beta^{(i)} &= 0 \quad \text{on } \partial\Omega \\ \nabla^2 \beta^{(i)} &= 0 \quad \text{in } \Omega \\ \nabla^2 \gamma^{(i)} + Re(\mathbf{u}_f^{(i)} \cdot \nabla \gamma^{(i)}) - \beta^{(i)} &= 0 \quad \text{in } \Omega \\ \gamma^{(i)} &= 0 \quad \text{on } \partial\Omega \end{aligned}$$

where $i = 1, 2, \dots, n+1$. The resolution procedure with those equations can be summarized as the following steps:

1. Initialize the scalar fields and the velocity fields with Stokes solver.
2. Use the adjoint equations in the iteration to update the function $\gamma^{(i)}$ in each iteration.
3. Update the boundary vorticity with the descent gradient method

$$\omega_{BC}^{(i+1)} = \omega_{BC}^{(i)} + \lambda \nabla_{\eta} J^{(i)}$$

where λ is a relaxation number. The gradient of the objective functional at each iteration is given by

$$\nabla_{\eta} J^{(i)} = \alpha \eta^{(i)} - \frac{\partial \gamma^{(i)}}{\partial n}$$

4. Determine the value of $\omega^{(i+1)}$ and $\tilde{\psi}^{(i+1)}$, and update the frozen velocity.

Caution

This approach requires a solver of nonlinear convection-diffusion equation (because the velocity $\mathbf{u}$ also depends on the stream-function and the vorticity is its rotational), and we do not have one in Basilisk for now.

Weighted Integral Method

Idea

The steady state stream function-vorticity formulation with an auxiliary function ϕ is written as

$$\begin{cases} \nabla^2 \psi = \omega \\ \nabla^2 \omega - Re \nabla \cdot (\mathbf{u} \omega) = 0 \\ \nabla^2 \phi + Re \nabla \cdot (\mathbf{u} \phi) = 1 \end{cases}$$

The correction relation is

$$\int_{\partial\Omega} \frac{\partial \tilde{\psi}^{(i+1)}}{\partial n} - \frac{\partial \tilde{\psi}^{(i)}}{\partial n} d\Gamma = \int_{\partial\Omega} (\omega^{(i+1)} - \omega^{(i)}) \frac{\partial \phi^{(i)}}{\partial n} d\Gamma$$

The update scheme of the boundary vorticity is given by

$$\omega_{BC}^{(i+1)} = \omega_{BC}^{(i)} + \alpha \left(\frac{\partial \phi^{(i)}}{\partial n} \right)^{-1} \left(\frac{\partial \psi_{exact}}{\partial n} - \frac{\partial \tilde{\psi}^{(i)}}{\partial n} \right)$$

where α is a relaxation number. Since the convection-diffusion equation of ϕ in each iteration is

$$\nabla^2 \phi^{(i)} + Re \nabla \cdot (\mathbf{u}_0^{(i)} \phi^{(i)}) = 1$$

thus we need to solve this equation with time marching scheme. Let us consider the time

dependent equation of ϕ such that

$$\frac{\partial \phi}{\partial t} - \nabla \cdot (\mathbf{u}_0 \phi) = \nu \nabla^2 \phi - \nu$$

thus the discrete form can be written as

$$\frac{\phi^{(i,m+1)} - \phi^{(i,m)}}{\Delta t} - \nabla \cdot (\mathbf{u}_0^{(i)} \phi^{(i,m)}) = \nu \nabla^2 \phi^{(i,m+1)} - \nu$$

We can introduce an intermediate value ϕ^* , therefore the previous equation can be split into a system of equations

$$\begin{aligned} \frac{\phi^* - \phi^{(i,m)}}{\Delta t} - \nabla \cdot (\mathbf{u}_0^{(i)} \phi^{(i,m)}) &= 0 \\ \frac{\phi^{(i,m+1)} - \phi^*}{\Delta t} &= \nu \nabla^2 \phi^{(i,m+1)} - \nu \end{aligned}$$

and we will obtain

$$\begin{aligned} \phi^* &= \phi^{(i,m)} + \Delta t \nabla \cdot (\mathbf{u}_0 \phi^{(i,m)}) \\ \nu \Delta t \nabla^2 \phi^{(i,m+1)} - \phi^{(i,m+1)} &= \nu \Delta t - \phi^* \end{aligned}$$

For the first equation, we can use an advection-diffusion scheme to solve it, and the second equation is a Poisson equation.

The update scheme of the boundary vorticity is given by

$$\omega_{BC}^{(i+1)} = \omega_{BC}^{(i)} + \alpha \left(\frac{\partial \phi^{(i)}}{\partial n} \right)^{-1} \left(\frac{\partial \psi_{exact}}{\partial n} - \frac{\partial \tilde{\psi}^{(i)}}{\partial n} \right)$$

where α is a relaxation number. Therefore, the numerical scheme can be summarized as the following steps:

1. Initialize the flow with Stokes solver.
2. Use the time marching scheme to solve the vorticity transport equation with the frozen velocity given by the stream function in the previous iteration. Solve the Poisson equation of the stream function ψ . Compute its normal derivative.
3. Use the pseudo time marching scheme to solve the convection-diffusion equation of ϕ with the frozen velocity. Calculate its normal derivative on the boundary, then update the boundary vorticity. Iterate this step till the boundary error is small enough.
4. Update the frozen velocity, and redo the step 2 and step 3 till the boundary error is small.

Reference:

- Pieter Wesseling, *Principles of Computational Fluid Dynamics*, Springer Series in Computational Mathematics
- Max D. Gunzburger, *Perspectives in Flow Control and Optimization*, Advances in Design and Control, SIAM